

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Отчет о лабораторной работе № 3

по курсу «Операционные системы»

«Список процессов в системе»

Студент: Лавров Р. Д.

Группа: ИУ9-42Б

Преподаватель: Брагин А. В.

Москва 2026

1 Содержание

1. Постоновка задачи
2. Практическая реализация
3. Результаты
4. Выводы
5. Список литературы

2 Постоновка задачи

1 часть: На основе драйвера из лабораторной работы № 2 создать новый драйвер, который выводит в отладочный лог список процессов в системе, используя API ядра. Список должен состоять из имени процесса (имя образа — Image Name), идентификатора про-цесса (PID) и идентификатора родительского процесса. Драйвер должен загружаться и выгружаться по требованию так, чтобы можно было проверить соответствие списков процес-сов, выведенных драйвером, и с помощью утилиты Task Manager (taskmgr.exe).Информацию о процессах получить из ядра с использованием функции ZwQuerySystemInformation(SystemProcessInformation, . . .). Выделение и освобождение памяти для информации о процессах реализовать, используя функции ExAllocatePool(PagedPool, . . .) и ExFreePool(). Предусмотреть ситуацию, в которой запрошенный размер буфера, необходимый для размещения информации о про-цессах, может измениться в бî´льшую сторону между вызовами функции ZwQuerySystemInformation, и в этом случае произвести но-вое выделение памяти с освобождением старого буфера.

2 часть: На основе драйвера из лабораторной работы № 2 создать новый драйвер, который выводит в отладочный лог список процессов в системе, используя API ядра. Список должен состоять из имени процесса (имя образа — Image Name), идентификатора про-цесса (PID) и идентификатора родительского процесса. Драйвер должен загружаться и выгружаться по требованию

так, чтобы можно было проверить соответствие списков процессов, выведенных драйвером, и с помощью утилиты ps.Информацию о процессах получить из ядра, используя гло-бальную переменную allproc. Для этого необходимо включить заголовочные файлы:
#include <sys/module.h>,
#include <sys/kernel.h>,
#include <sys/proc.h>
Самостоятельно следует изучить проблему защиты от «грязно-го чтения» списка процессов в момент его изменения операцион-ной системой (например, при создании нового процесса или уда-лении завершенного).

3 Практическая реализация

3.1 ReactOS

Первоначально в файл drivers/CMakelists.txt была добавлена следующая строка:

```
add_subdirectory(lab3)
```

Затем в каталоге drivers создана папка lab3, в которую помещены файлы lab3.c, lab3.rc и CMakeLists.txt.

Листинг 1: CMakeLists.txt

```
add_library(lab3 MODULE
    lab3.c
    lab3.rc
)
set_module_type(lab3 kernelmodedriver)
add_importlibs(lab3 ntoskrnl hal)
add_cd_file(TARGET lab3 DESTINATION reactos/system32/drivers FOR all)
```

Листинг 2: lab3.rc

```
#define REACTOS_VERSION_DLL
#define REACTOS_STR_FILE_DESCRIPTION  "Lab3 Driver"
#define REACTOS_STR_INTERNAL_NAME     "lab3"
#define REACTOS_STR_ORIGINAL_FILENAME "lab3.sys"
```

```
#include <reactos/version.rc>
```

Листинг 3: lab3.c

```
#include <ntddk.h>
#include <debug.h>
#include <ntifs.h>
#include <ndk/exfuncs.h>
#include <ndk/ketypes.h>
#include <pseh/pseh2.h>
#include <ntstrsafe.h>

PDEVICE_OBJECT gDeviceObject = NULL;

VOID PrintProcessList(VOID)
{
    NTSTATUS status;
    ULONG neededSize = 0;
    PSYSTEM_PROCESS_INFORMATION pInfo = NULL;
    PSYSTEM_PROCESS_INFORMATION pCurrent = NULL;

    status = ZwQuerySystemInformation(SystemProcessInformation, NULL, 0, &neededSize);
    if (status != STATUS_INFO_LENGTH_MISMATCH) {
        DPRINT1("ZwQuerySystemInformation (size) failed: 0x%08lx\n", status);
        return;
    }

    pInfo = (PSYSTEM_PROCESS_INFORMATION)ExAllocatePoolWithTag(
        PagedPool, neededSize, 'tpSL');
    if (!pInfo) {
        DPRINT1("Failed to allocate memory for process list\n");
        return;
    }

    status = ZwQuerySystemInformation(SystemProcessInformation, pInfo, neededSize, &
    neededSize);
    if (!NT_SUCCESS(status)) {
        DPRINT1("ZwQuerySystemInformation (data) failed: 0x%08lx\n", status);
        ExFreePoolWithTag(pInfo, 'tpSL');
        return;
    }

    DPRINT1("\n=== Process list (PID, PPID, Image name) ===\n");

    pCurrent = pInfo;
    while (TRUE) {
```

```
42          if (pCurrent->ImageName.Buffer && pCurrent->ImageName.Length > 0) {
43              DPRINT1("PID: %5lu  PPID: %5lu  Image: %wZ\n",
44                      (ULONG_PTR)pCurrent->UniqueProcessId,
45                      (ULONG_PTR)pCurrent->InheritedFromUniqueProcessId,
46                      &pCurrent->ImageName);
47          } else {
48              DPRINT1("PID: %5lu  PPID: %5lu  Image: <system process>\n",
49                      (ULONG_PTR)pCurrent->UniqueProcessId,
50                      (ULONG_PTR)pCurrent->InheritedFromUniqueProcessId);
51          }
52
53          if (pCurrent->NextEntryOffset == 0)
54              break;
55          pCurrent = (PSYSTEM_PROCESS_INFORMATION)((PUCHAR)pCurrent + pCurrent
    ->NextEntryOffset);
56      }
57
58      ExFreePoolWithTag(pInfo, 'tpSL');
59  }
60
61  VOID NTAPI DriverUnload(IN PDRIVER_OBJECT DriverObject)
62  {
63      UNREFERENCED_PARAMETER(DriverObject);
64
65      if (gDeviceObject)
66      {
67          IoDeleteDevice(gDeviceObject);
68          gDeviceObject = NULL;
69      }
70
71      DPRINT1("Driver unloaded\n");
72  }
73
74  NTSTATUS NTAPI DriverEntry(
75      IN PDRIVER_OBJECT DriverObject,
76      IN PUNICODE_STRING RegistryPath)
77  {
78      UNREFERENCED_PARAMETER(RegistryPath);
79
80      DriverObject->DriverUnload = DriverUnload;
81
82      DPRINT1("Driver Lavrov\n");
83
84      UNICODE_STRING devName = RTL_CONSTANT_STRING(L"\\Device\\lab2dev");
85      NTSTATUS status = IoCreateDevice(
86          DriverObject,
```

```
87              0,
88              &devName,
89              FILE_DEVICE_UNKNOWN,
90              0,
91              FALSE,
92              &gDeviceObject
93      );
94
95      if (!NT_SUCCESS(status))
96          DPRINT1("Failed to create device object\n");
97
98      PrintProcessList();
99
100     return STATUS_SUCCESS;
101 }
```

С использованием команды ninja bootcd был собран ISO-образ, который впоследствии установлен в QEMU. После этого в ReactOS открыта консоль, и драйвер загружен следующими командами:

```
1 sc create lab3driver binPath= C:\ReactOS\system32\drivers\ lab3.sys type= kernel
2 sc start lab3driver
```

3.2 NetBSD

Для реализации драйвера в NetBSD создан файл lab3.c в каталоге usr/src/sys/dev.

Листинг 4: lab3.c

```
1  #include <sys/module.h>
2  #include <sys/systm.h>
3  #include <sys/proc.h>
4  #include <sys/mutex.h>
5
6  MODULE(MODULE_CLASS_MISC, lab3, NULL);
7
8   static  void
9  print_process_list(void)
10 {
11      struct proc *p;
12
```

```
13      mutex_enter(&proc_lock);
14      PROCLIST_FOREACH(p, &allproc) {
15              printf("PID: %-5d PPID: %-5d Name: %s\n",
16                      p->p_pid,
17                      p->p_pptr ? p->p_pptr->p_pid : -1,
18                      p->p_comm);
19      }
20      mutex_exit(&proc_lock);
21  }
22
23  static  int
24  lab3_modcmd(modcmd_t cmd, void *arg)
25  {
26      switch (cmd) {
27          case MODULE_CMD_INIT:
28              printf("Lab3 driver loaded: Lavrov\n");
29              print_process_list();
30              return 0;
31
32          case MODULE_CMD_FINI:
33              printf("Lab3 driver unloaded: Lavrov\n");
34              return 0;
35
36          default:
37              return ENOTTY;
38      }
39  }
```

Далее в директории /usr/src/sys/modules была создана папка lab3, в которую добавлен файл Makefile.

Листинг 5: Makefile

```
1  .include "../Makefile.inc"
2  KMOD=lab3
3  .PATH: ${S}/dev
4  SRCS=lab3.c
5  .include <bsd.kmodule.mk>
```

Затем модуль был скомпилирован командой make и загружен с помощью modload lab3.

4 Результаты

4.1 ReactOS

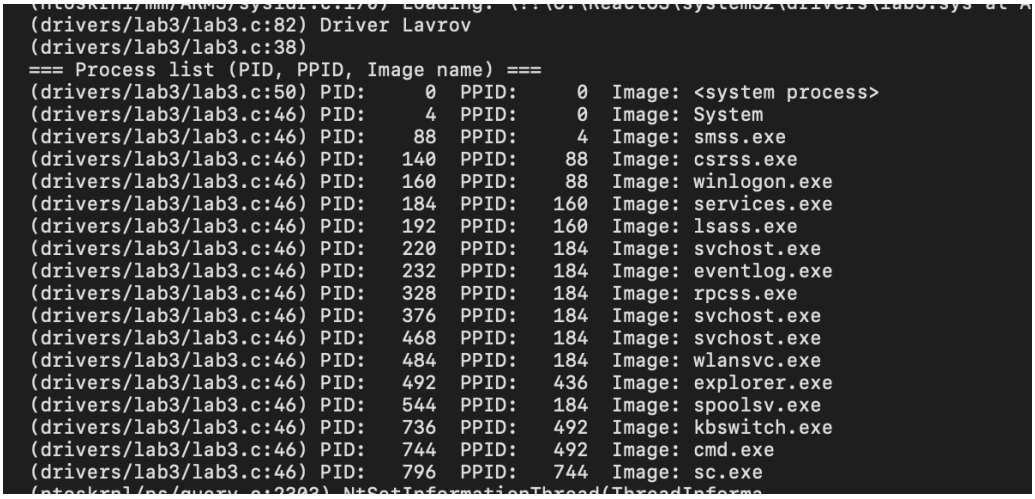


Рис. 1 — Результат работы

4.2 NetBSD

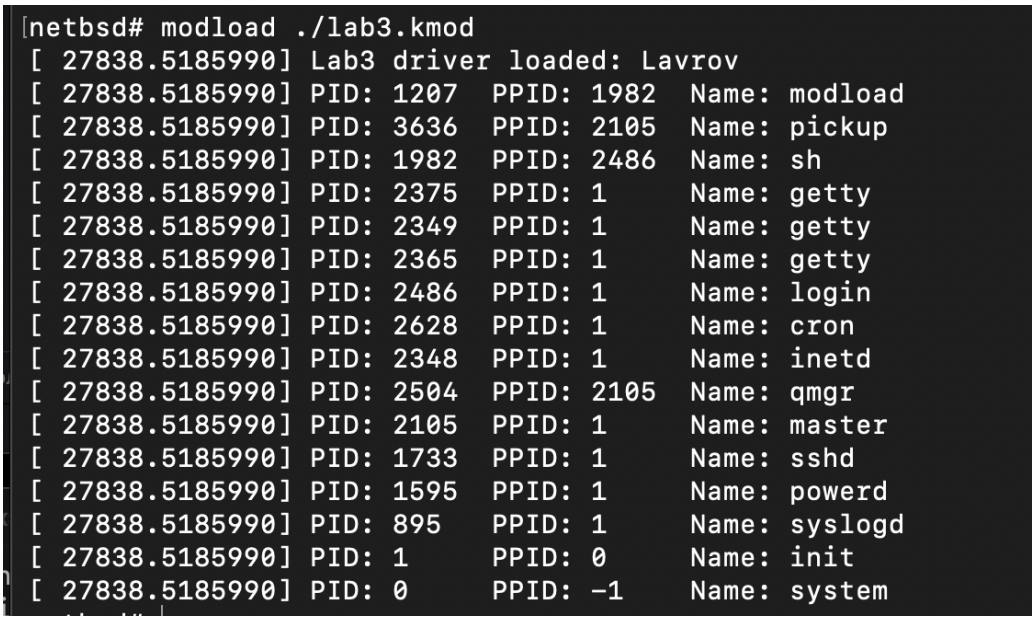


Рис. 2 — Результат работы

5 Выводы

В ходе работы разработаны драйверы для ReactOS и NetBSD, выводящие список активных процессов (PID, PPID, имя образа) в отладочный лог. Для ReactOS использовано API ‘ZwQuerySystemInformation‘ с динамическим выделением памяти и обработкой изменения размера буфера. Для NetBSD применён обход глобального списка ‘allproc‘. Драйверы успешно собраны, загружены, их вывод соответствует данным диспетчера задач (Task Manager) и утилиты ‘ps‘.

6 Список литературы

1. Учебно-методическое пособие - https://press.bmstu.ru/catalog/item/8226/reader/
2. ReactOS – https://reactos.org
3. ReactOS Build Environment (RosBE) Documentation – https://reactos.org/wiki/Build_Environment
4. NetBSD Project – https://www.netbsd.org
5. The NetBSD Guide – https://www.netbsd.org/docs/guide/en/
6. QEMU Documentation – https://www.qemu.org/docs/master/